

Part 4: Cybersecurity

Chapter 1: The Suspicious Email

Monday morning began like any other.

The engineering team gathered for their daily stand-up.

Maya looked at the sprint board.

> "This week's goal is to finish the new billing dashboard."

Everyone nodded.

Just as Alex opened his laptop, a notification appeared.

****New Email****

From: IT Support

Subject: ****URGENT: Verify Your Account Immediately****

Alex clicked it.

The email claimed that his account would be suspended unless he verified his password within the next thirty minutes.

He moved his mouse toward the ****Verify Now**** button.

"Wait."

Sarah, the company's security engineer, had just walked into the room.

> "Don't click it."

Alex looked surprised.

> "Why?"

Sarah smiled.

> "Because it's a ****phishing email****."

Looking for Red Flags

Sarah projected the email onto the screen.

> "Let's investigate."

Emma noticed something immediately.

> "The sender's address looks strange."

Instead of:

****[it-support@ourcompany.com] (mailto:it-support@ourcompany.com) ****

it said:

****[it-support@ourcompany.com] (mailto:it-support@ourcompany.com) ****

David laughed.

> "The letter 'm' was replaced with 'rn'."

Sarah nodded.

> "That's called ****typosquatting****."

She continued.

> "Attackers register domains that closely resemble legitimate ones."

Social Engineering

Maya read the email carefully.

> "It says my account will be deleted in thirty minutes."

Sarah nodded.

> "That's intentional."

She wrote two words on the whiteboard.

****Social Engineering****

Emma asked,

> "What's that?"

Sarah explained,

> "Instead of attacking computers, attackers manipulate people."

Alex nodded.

> "They're creating a false sense of urgency."

> "Exactly."

Credential Theft

David pointed to the fake login page.

> "If Alex entered his password, what would happen?"

Sarah replied,

> "The attacker would steal his ****credentials****."

Emma frowned.

> "Then they could log into our systems."

Sarah nodded.

> "That's called **credential theft**."

Multi-Factor Authentication

Alex smiled.

> "Even if they had my password, wouldn't they still need my second factor?"

Sarah replied,

> "Exactly."

She wrote another term.

Multi-Factor Authentication (MFA)

> "Authentication should require at least two independent factors."

She listed them.

- * Something you know (password)
- * Something you have (security key or phone)
- * Something you are (fingerprint or face scan)

Emma smiled.

> "One stolen password isn't enough."

Password Hygiene

James admitted something.

> "I use the same password for several websites."

The room became quiet.

Sarah shook her head.

> "That's called **password reuse**."

She explained,

> "If one website is compromised, attackers will try the same password everywhere else."

David nodded.

> "That's called **credential stuffing**, right?"

> "Exactly."

Sarah continued.

> "Every account should have a unique password."

Password Managers

Emma asked,

> "How can anyone remember hundreds of unique passwords?"

Sarah smiled.

> "You don't."

She introduced another tool.

> "Use a **password manager**."

Alex nodded.

> "Then I only need to remember one strong master password."

Least Privilege

Maya changed the topic.

> "Suppose an attacker compromises one employee's account."

Sarah asked,

> "Should that account have access to every system?"

Everyone answered together.

> "No."

Sarah smiled.

> "That's the **principle of least privilege**."

She explained,

> "Every user should receive only the permissions necessary to perform their job."

Privilege Escalation

Chris raised another question.

> "What if an attacker starts with a normal user account?"

Sarah replied,

> "They'll often attempt **privilege escalation**."

Emma asked,

> "Meaning they try to become an administrator?"

> "Exactly."

Security Awareness

Sarah closed the fake email.

> "Technology alone isn't enough."

She looked around the room.

> "The strongest firewall in the world can't stop someone from voluntarily giving away their password."

Alex laughed.

> "So people are part of the security system."

Sarah smiled.

> "Exactly."

Reporting Suspicious Emails

Later that afternoon, another suspicious email arrived.

This time, nobody clicked the link.

Emma used the company's **Report Phishing** button.

Within minutes, the security team analyzed the message.

Chris smiled.

> "The email was blocked for everyone else before anyone else opened it."

The Security Team's Investigation

Sarah opened the email headers.

She examined:

- * Sender domain
- * Return path
- * SPF
- * DKIM
- * DMARC

Alex looked curious.

> "Those look complicated."

Sarah replied,

> "We'll study them later."

She smiled.

> "For today, just remember that they help verify whether an email is legitimate."

Wrapping Up

Before leaving the meeting room, Maya summarized the day's lessons.

She wrote a checklist.

- ☒ Phishing
- ☒ Typosquatting
- ☒ Social Engineering
- ☒ Credential Theft
- ☒ Multi-Factor Authentication (MFA)
- ☒ Password Reuse
- ☒ Credential Stuffing
- ☒ Password Manager
- ☒ Least Privilege
- ☒ Privilege Escalation
- ☒ Security Awareness

Emma closed her laptop.

> "This morning I thought cybersecurity was mostly about hackers."

Sarah smiled.

> "Today you learned that attackers often begin by targeting people."

Alex nodded.

> "The first line of defense isn't software."

He smiled.

> "It's awareness."

The team returned to work knowing that one careful decision—not clicking a suspicious link—could prevent an entire security incident.

Part 4: Cybersecurity

Chapter 2: The Laptop That Wouldn't Behave

Tuesday morning started with an unusual message.

James walked into the office carrying his laptop.

He looked worried.

> "Something isn't right."

Emma looked up.

> "What happened?"

James sighed.

> "My laptop has become incredibly slow."

He continued,

> "The cooling fan is running constantly."

> "Applications keep freezing."

> "And I noticed network activity even when I wasn't doing anything."

Sarah, the security engineer, immediately became serious.

> "Disconnect it from the network."

James unplugged the Ethernet cable and turned off Wi-Fi.

Alex looked confused.

> "Isn't that a bit extreme?"

Sarah shook her head.

> "If the laptop is compromised, isolating it helps prevent the threat from spreading."

Malware

Sarah connected the laptop to an isolated analysis network.

She looked at the team.

> "Today we're investigating possible **malware**."

Emma asked,

> "Malware is just another word for a virus, right?"

Sarah smiled.

> "Not exactly."

She wrote a word on the whiteboard.

Malware

> "Malware is a general term for any malicious software."

She continued,

> "Viruses are only one type of malware."

Virus

Alex asked,

> "What makes a virus different?"

Sarah explained,

> "A **virus** attaches itself to another file or program."

> "It usually requires someone to execute the infected file before it spreads."

Emma nodded.

> "So the user accidentally helps the virus."

> "Exactly."

Worm

David asked,

> "What about a worm?"

Sarah replied,

> "A **worm** spreads automatically across networks."

She continued,

> "Unlike a virus, it doesn't need a user to open an infected file."

James frowned.

> "That sounds much more dangerous."

Sarah nodded.

> "It often is."

Trojan

Emma remembered another term.

> "What's a Trojan?"

Sarah smiled.

> "A **Trojan horse** pretends to be legitimate software."

She explained,

> "The user installs it willingly because it appears harmless."

Alex laughed.

> "Just like the story from ancient Greece."

> "Exactly."

Spyware

James opened the Task Manager.

Several unfamiliar processes were running.

Sarah investigated.

> "This could be **spyware**."

Emma asked,

> "What does spyware do?"

Sarah answered,

> "It secretly collects information."

> "It may record keystrokes, monitor browsing activity, or steal sensitive data."

Ransomware

David looked concerned.

> "I've heard about ransomware attacks."

Sarah nodded.

She displayed a screenshot from a real incident.

****Your files have been encrypted.****

****Pay within 72 hours.****

Emma looked shocked.

Sarah explained,

> **"**Ransomware** encrypts a victim's files and demands payment for the decryption key."**

Alex asked,

> "Should companies pay?"

Sarah replied,

> "Most security experts recommend restoring from backups instead of paying criminals."

Antivirus

Chris launched the company's ****antivirus**** software.

It immediately detected suspicious files.

James smiled.

> "Problem solved?"

Sarah shook her head.

> "Not necessarily."

She explained,

> "Traditional antivirus mainly detects known threats."

> "Modern attacks often require additional protection."

Endpoint Detection and Response

Sarah opened another dashboard.

Emma noticed unfamiliar graphs.

> "What's this tool?"

Sarah replied,

> **"**Endpoint Detection and Response**, or ****EDR****."**

She continued,

> "Instead of only detecting malware, EDR continuously monitors endpoint behavior."

Alex nodded.

> "So it looks for suspicious activity, not just known virus signatures."

> "Exactly."

Indicators of Compromise

The EDR platform highlighted several alerts.

Sarah pointed to them.

> "These are **Indicators of Compromise**, often called **IOCs**."

Emma asked,

> "Examples?"

Sarah listed a few.

- * Unknown processes
- * Suspicious network connections
- * Unexpected file modifications
- * Communication with malicious IP addresses

James nodded.

> "My laptop shows several of those."

Zero-Day Vulnerability

Chris reviewed the security alerts.

One message caught his attention.

> "A new vulnerability was published this morning."

Sarah looked carefully.

> "It's a **zero-day vulnerability**."

Emma asked,

> "Why is it called zero-day?"

Sarah explained,

> "Because the software vendor has had zero days to create a patch before attackers begin exploiting it."

The room became quiet.

CVE

Sarah searched the vulnerability database.

She found an entry.

CVE-2026-18457

Alex asked,

> "What's a CVE?"

Sarah replied,

> **"Common Vulnerabilities and Exposures"**.

She continued,

> "Each publicly disclosed vulnerability receives a unique CVE identifier."

Emma smiled.

> "Like tracking numbers for security issues."

Patch Management

Chris checked the company's inventory.

James's laptop had missed the latest security update.

Sarah sighed.

> "That's why **patch management** is so important."

Emma nodded.

> "Keeping software updated reduces the number of exploitable vulnerabilities."

Quarantine

The antivirus finished scanning.

It displayed:

Threat Quarantined

David asked,

> "What does quarantine mean?"

Sarah replied,

> "The infected files are isolated so they can't execute or spread."

Digital Forensics

Even though the malware had been removed, Sarah wasn't finished.

Alex looked surprised.

> "Why keep investigating?"

Sarah smiled.

> "Because removing malware isn't enough."

She explained,

> "We also need to understand how it entered the system."

The team began a ****digital forensics**** investigation.

They examined:

- * Browser history
- * Download history
- * System logs
- * Network connections
- * Running processes

Finally, James remembered something.

> "Yesterday I downloaded a free PDF converter."

Sarah nodded.

> "That installer contained the Trojan."

Lessons Learned

Fortunately, the malware had been detected before it spread to other systems.

The company lost no data.

No backups were needed.

No ransom was paid.

Before ending the day, Maya summarized the incident.

She wrote another checklist.



Malware



Virus



Worm

- ✓ Trojan
- ✓ Spyware
- ✓ Ransomware
- ✓ Antivirus
- ✓ Endpoint Detection and Response (EDR)
- ✓ Indicators of Compromise (IOCs)
- ✓ Zero-Day Vulnerability
- ✓ CVE
- ✓ Patch Management
- ✓ Quarantine
- ✓ Digital Forensics

She looked around the room.

> "Today's incident wasn't caused by sophisticated hacking."

She smiled gently.

> "It started with one unsafe software download."

James nodded.

> "Cybersecurity isn't only about defending against attackers."

He closed his laptop.

> "It's also about making smart everyday decisions."

As everyone returned to work, the team realized that protecting a company often begins long before an attacker reaches the network—it begins with the choices each employee makes every day.

Part 4: Cybersecurity

Chapter 3: The Security Code Review

Wednesday morning began with an unexpected meeting invitation.

****Subject:** Security Review Before Production Release**

The engineering team gathered in the conference room.

Sarah connected her laptop to the projector.

> "Today's feature is complete."

She smiled.

> "Now let's see whether it's secure."

Alex laughed.

> "It passed all the unit tests."

Emma added,

> "Integration tests and E2E tests passed too."

Sarah nodded.

> "That's excellent."

She paused.

> "But secure software requires more than functional testing."

A Penetration Test

Sarah introduced a guest.

Daniel was an external security consultant.

> "Today I'll perform a **penetration test**."

David asked,

> "So you're going to attack our application?"

Daniel smiled.

> "With permission."

Everyone laughed.

> "That's exactly what a penetration test is."

OWASP Top 10

Before starting, Sarah wrote a title on the whiteboard.

OWASP Top 10

Emma asked,

> "I've heard of OWASP."

Sarah nodded.

> **"**OWASP (Open Worldwide Application Security Project)** publishes the most common web application security risks."**

Alex smiled.

> "So it's a great checklist during development."

SQL Injection

Daniel opened the login page.

Instead of typing a normal username, he entered:

```
```sql id="8gbgoh"
' OR '1'='1
```
```

Emma looked confused.

The application responded with an error.

Daniel smiled.

> "This is a classic ****SQL Injection**** attempt."

Chris asked,

> "How does it work?"

Daniel explained,

> "If an application directly inserts user input into an SQL query, attackers can change the query itself."

He showed an unsafe example.

```
```sql id="b2b3cm"
SELECT * FROM users
WHERE username = '<user_input>';
```
```

Emma frowned.

> "That looks dangerous."

Prepared Statements

Chris smiled.

> "Fortunately, we don't build SQL queries that way."

He opened the backend code.

> "We use ****prepared statements****."

Sarah nodded.

> "Prepared statements separate SQL code from user input."

Daniel smiled.

> "That's one of the best defenses against SQL Injection."

Input Validation

Alex examined another form.

> "Users can upload profile information."

Sarah asked,

> "Do we validate every field?"

Emma nodded.

> "We check length, format, required fields, and allowed characters."

Sarah smiled.

> "Excellent."

> "**Input validation** reduces many types of attacks."

Cross-Site Scripting (XSS)

Daniel created another test account.

This time, instead of a normal name, he entered:

```
```html id="tk9gxv"
<script>alert("Hello")</script>
```
```

When another user viewed the profile, the browser executed the script.

Emma looked surprised.

> "The browser actually ran it!"

Sarah nodded.

> "That's **Cross-Site Scripting**, or **XSS**."

Output Encoding

Chris immediately fixed the issue.

Instead of displaying raw HTML, the application escaped special characters.

Emma refreshed the page.

Now the browser displayed:

```
` ``text id="0svmrq"
<script>alert("Hello")</script>
` ``
```

as plain text.

Sarah smiled.

> **"**Output encoding** prevents browsers from interpreting untrusted input as executable code."**

Cross-Site Request Forgery (CSRF)

Daniel opened another demonstration.

He explained,

> "Imagine you're logged into your bank."

Then he visited a malicious website.

The page silently submitted a money transfer request.

Emma looked shocked.

> "The bank thinks I made the request."

Sarah nodded.

> "That's ****Cross-Site Request Forgery****, or ****CSRF****."

Chris smiled.

> "Our application uses ****CSRF tokens****."

Daniel nodded.

> "Then the attack fails."

Remote Code Execution

David asked,

> "What's one of the most dangerous vulnerabilities?"

Daniel answered immediately.

> **"**Remote Code Execution****, often called ****RCE****."

The room became quiet.

He explained,

> "If attackers achieve RCE, they can execute arbitrary code on your server."

Emma whispered,

> "That sounds catastrophic."

Sarah nodded.

> "It usually is."

File Upload Validation

Alex demonstrated the document upload feature.

Sarah asked,

> "Can users upload anything?"

Emma replied,

> "Only PDF files."

Daniel smiled.

> "How do you verify that?"

Chris answered,

> "We check the file extension."

Daniel shook his head.

> "That's not enough."

He renamed a malicious executable from:

```
```text id="3zlbdv"
virus.exe
```
```

to

```
```text id="2l36k2"
document.pdf
```
```

Emma laughed.

> "Good point."

Sarah added,

> "Always verify the file's actual content, not just its name."

Security Headers

Chris opened the browser.

He inspected the HTTP response headers.

Several important ones appeared.

- * Content-Security-Policy
- * X-Content-Type-Options
- * X-Frame-Options
- * Strict-Transport-Security

Emma asked,

> "What do these do?"

Sarah explained,

> "These ****security headers**** help browsers defend against common attacks."

Secure-by-Design

The penetration test ended successfully.

Daniel smiled.

> "Your application is in good shape."

Maya asked,

> "What's the biggest reason?"

Daniel replied,

> "You considered security during development."

Sarah wrote another phrase on the whiteboard.

****Secure-by-Design****

She explained,

> "Instead of adding security at the end, we build it into every stage of development."

Defense in Depth

Alex asked,

> "If prepared statements already stop SQL Injection, why do we also validate input?"

Sarah smiled.

> "Because security should never depend on a single defense."

She wrote another principle.

****Defense in Depth****

> "If one layer fails, another layer should still protect the system."

Emma nodded.

> "Multiple safeguards working together."

Final Review

Before ending the meeting, Maya summarized the security review.

She wrote one final checklist.



Penetration Test



OWASP Top 10



SQL Injection



Prepared Statements



Input Validation



Cross-Site Scripting (XSS)



Output Encoding



Cross-Site Request Forgery (CSRF)



CSRF Token



Remote Code Execution (RCE)



File Upload Validation



Security Headers



Secure-by-Design



Defense in Depth

She looked around the room.

> "Today's review wasn't about finding mistakes."

She smiled.

> "It was about preventing future incidents."

Daniel packed his laptop.

> "The best security vulnerability is the one that never reaches production."

Emma closed her editor.

> "Writing code is only half the job."

Alex nodded.

> "Writing secure code is the other half."

The team left the conference room confident that every feature they built would now be stronger, safer, and more resilient against real-world attacks.

Part 4: Cybersecurity

Chapter 4: The Midnight Security Incident

It was 2:13 AM.

Emma's phone vibrated on the bedside table.

****🔴 Critical Security Alert****

She opened the notification.

****Multiple failed login attempts detected.****

A few seconds later, another message arrived.

****Potential compromise detected.****

Emma immediately joined the emergency video call.

Within minutes, Maya, Sarah, Chris, Alex, James, and David were online.

Maya looked serious.

> "This isn't a drill."

> "Let's begin our ****incident response****."

The Security Operations Center

Sarah shared her screen.

A large dashboard displayed alerts from across the company's infrastructure.

Alex asked,

> "Is this our SOC dashboard?"

Sarah nodded.

> "***SOC** stands for **Security Operations Center**."

She explained,

> "The SOC continuously monitors systems for security threats."

SIEM

The dashboard was filled with logs from servers, firewalls, cloud services, and applications.

Emma asked,

> "How are all these logs collected in one place?"

Sarah replied,

> "Through our **SIEM**."

She wrote the full name on the whiteboard.

****Security Information and Event Management****

Chris added,

> "The SIEM aggregates logs, correlates events, and generates alerts."

A Suspicious Pattern

James zoomed in on the dashboard.

Thousands of failed login attempts appeared.

All targeted the administrator portal.

David frowned.

> "That's definitely unusual."

Sarah nodded.

> "The SIEM correlated multiple events and raised a high-priority alert."

Intrusion Detection System

Chris checked another dashboard.

> "Our **Intrusion Detection System (IDS)** noticed unusual authentication traffic about twenty minutes ago."

Emma asked,

> "What does an IDS actually do?"

Sarah explained,

> "It detects suspicious activity and alerts security teams."

Alex nodded.

> "But it doesn't block the traffic."

> "Correct."

Intrusion Prevention System

James looked at another panel.

> "Our **Intrusion Prevention System (IPS)** has already blocked several malicious IP addresses."

Emma smiled.

> "So the IPS goes one step further."

Sarah nodded.

> "An IPS doesn't just detect attacks."

> "It actively prevents them."

Brute Force Attack

David studied the authentication logs.

> "The attacker is trying thousands of passwords."

Sarah replied,

> "That's a **brute force attack**."

Chris added,

> "Fortunately, our rate limiting and account lockout policies slowed it down."

Threat Intelligence

Sarah opened another platform.

Emma noticed a long list of IP addresses.

> "Where did those come from?"

Sarah answered,

> "***Threat intelligence** feeds."

She explained,

> "Security organizations continuously share information about known malicious infrastructure."

Alex smiled.

> "So we already know many dangerous IP addresses."

Indicators of Attack

James noticed another alert.

> "The attacker changed tactics."

Sarah pointed at the timeline.

> "These behaviors are called **Indicators of Attack (IOAs)**."

Emma asked,

> "How are they different from Indicators of Compromise?"

Sarah explained,

> "***IOCs** show evidence that an attack has already happened."

> "***IOAs** describe suspicious behavior while an attack is unfolding."

Alex nodded.

> "That gives us a chance to stop the attack earlier."

Correlation

The SIEM connected multiple events automatically.

- * Failed logins
- * Suspicious IP addresses
- * Unusual API requests
- * Geographic anomalies

Chris smiled.

> "That's **event correlation**."

Emma asked,

> "Instead of analyzing each alert separately, the SIEM combines them."

> "Exactly."

Containment

Maya looked around the room.

> "First priority?"

Sarah answered immediately.

> "***Containment**."

The team quickly:

- * Blocked malicious IP addresses.
- * Disabled the targeted administrator account.
- * Rotated API keys.
- * Restricted external access.

Emma nodded.

> "We're preventing the attack from spreading."

Eradication

Once the attack had been contained, Sarah continued.

> "Now we move to **eradication**."

James searched every affected server.

No malware was found.

Chris removed several outdated administrator accounts.

David revoked old authentication tokens.

Sarah smiled.

> "We're eliminating every trace of the attack."

Recovery

Alex asked,

> "Can we reopen the administrator portal?"

Sarah reviewed the dashboards carefully.

Everything looked normal.

> "Yes."

The team restored normal operations.

Maya smiled.

> "That's the ****recovery**** phase."

Root Cause Analysis

The immediate danger had passed.

Now the team wanted answers.

Sarah reviewed the logs.

A few minutes later, she smiled.

> "I found the root cause."

Emma leaned forward.

> "What was it?"

> "One old administrator account still existed."

James looked surprised.

> "Nobody was using it."

Sarah nodded.

> "Exactly."

> "Inactive accounts are still security risks."

Lessons Learned

The next morning, the entire engineering department gathered for a review.

Maya began.

> "What worked well?"

Emma replied,

> "The SIEM detected suspicious activity quickly."

Alex added,

> "Threat intelligence identified known attackers."

James continued,

> "The IPS blocked malicious traffic automatically."

David smiled.

> "Containment prevented the attack from spreading."

Sarah nodded.

> "Excellent."

Action Items

Before ending the meeting, Maya summarized the improvements.

- * Remove inactive accounts.
- * Enforce Multi-Factor Authentication for all administrators.
- * Review firewall rules.
- * Rotate credentials regularly.
- * Improve monitoring dashboards.
- * Schedule quarterly penetration tests.
- * Update the incident response playbook.

Everyone agreed.

The Incident Response Lifecycle

Sarah drew one final diagram.

****Preparation****

↓

****Detection****

↓

****Analysis****

↓

****Containment****

↓

****Eradication****

↓

****Recovery****

↓

****Lessons Learned****

She smiled.

> "Cybersecurity isn't about assuming attacks will never happen."

She looked around the room.

> "It's about responding quickly, minimizing damage, and learning from every incident."

Alex closed his laptop.

> "Tonight proved something important."

Emma smiled.

> "Security isn't one product."

James nodded.

> "It's a continuous process."

As the sun rose outside the office windows, the team realized that the true strength of a security program isn't measured by the number of attacks it prevents—but by how effectively it detects, contains, and recovers from the attacks that inevitably occur.

Part 4: Cybersecurity

Chapter 5: Following the Digital Evidence

The security incident was over.

The attackers had been blocked.

The systems were stable again.

Most of the engineering team returned to their normal work.

Sarah didn't.

She walked into the conference room carrying a sealed evidence bag.

Emma looked surprised.

> "The attack is over."

Sarah nodded.

> "The attack is over."

She placed the bag on the table.

> "The investigation is just beginning."

A Possible Data Breach

Maya entered the room.

> "One customer contacted support this morning."

Emma looked concerned.

> "What happened?"

> "They believe someone accessed their account."

The room became quiet.

Sarah spoke carefully.

> "We need to determine whether this was a ****data breach****."

Digital Forensics

Sarah connected a dedicated forensic workstation.

Alex asked,

> "Why not investigate directly on the employee's laptop?"

Sarah shook her head.

> "Because we might accidentally modify evidence."

She wrote two words on the whiteboard.

****Digital Forensics****

> "Our goal is to investigate without changing the original evidence."

Chain of Custody

Sarah handed James a document.

Emma looked at it.

Every person who handled the laptop had signed it.

David asked,

> "Why is that necessary?"

Sarah replied,

> "This is the ****chain of custody****."

She explained,

> "It documents who collected, transported, stored, and analyzed the evidence."

Alex nodded.

> "So the evidence remains trustworthy."

Forensic Imaging

Instead of opening the laptop's hard drive, Sarah connected a specialized device.

James looked confused.

> "Why aren't we examining the original disk?"

Sarah smiled.

> "Because we never investigate the original."

She created a **forensic image**.

Emma asked,

> "That's an exact copy?"

> "Bit for bit."

Hash Values

After the copy finished, Sarah generated two long strings.

Alex asked,

> "What are those?"

Sarah replied,

> **"Cryptographic hashes"**.

She generated a SHA-256 hash for both the original disk and the forensic image.

The values matched exactly.

Emma smiled.

> "So the copy is identical."

Sarah nodded.

> "If even one bit changed, the hash would be completely different."

Audit Logs

Chris opened the company's logging platform.

Sarah asked,

> "What should we check next?"

Emma answered,

> "***Audit logs**."

Chris smiled.

> "Exactly."

The logs showed:

- * User logins
- * Permission changes
- * File downloads
- * Administrative actions

David nodded.

> "Everything important is recorded."

Non-Repudiation

Alex noticed that every administrative action included:

- * User ID
- * Timestamp
- * IP address
- * Digital signature

Sarah smiled.

> "That's **non-repudiation**."

Emma asked,

> "Meaning users can't deny performing an action?"

> "Exactly."

Timeline Analysis

James arranged every event chronologically.

9:42 AM

User logged in.

9:45 AM

Customer record viewed.

9:47 AM

Permission denied.

9:48 AM

Administrator login.

9:49 AM

Password reset.

Sarah nodded.

> "Building a timeline often reveals what really happened."

Data Classification

Maya joined the discussion.

She asked,

> "Was any sensitive information exposed?"

Sarah opened the company's data classification policy.

Every type of information had a label.

- * Public
- * Internal
- * Confidential
- * Restricted

Emma smiled.

> "Customer payment information is Restricted."

Chris added,

> "Marketing documents are Public."

Sarah nodded.

> "Different classifications require different protections."

Data Retention

David noticed another policy.

> "Why do we still have logs from last year?"

Sarah explained,

> "Because of our **data retention** policy."

She continued,

> "Some logs must be kept for several years."

Emma asked,

> "Can we keep everything forever?"

Sarah shook her head.

> "No."

> "Keeping unnecessary data also creates security and privacy risks."

GDPR

The legal team joined the meeting.

Their first question was simple.

> "Are any European customers affected?"

Sarah replied,

> "We're still investigating."

The lawyer nodded.

> "If personal information belonging to EU residents was exposed,
GDPR reporting requirements may apply."

Emma asked,

> "So cybersecurity also involves legal responsibilities."

> "Very much so."

PCI DSS

James reviewed the payment system.

Chris smiled.

> "Fortunately, we never store raw credit card numbers."

Sarah nodded.

> "That's part of **PCI DSS** compliance."

Emma asked,

> "The Payment Card Industry Data Security Standard?"

> "Exactly."

ISO 27001

Maya pointed toward another document.

> "Our security management system follows **ISO 27001**."

David asked,

> "Is that a law?"

Sarah replied,

> "No."

> "It's an internationally recognized information security standard."

Compliance Audit

Later that afternoon, external auditors arrived.

Emma looked nervous.

Sarah smiled.

> "Don't worry."

> "A **compliance audit** isn't about finding someone to blame."

She continued,

> "It's about verifying that our processes match our documented policies."

The Investigation Ends

After several hours, Sarah finished reviewing the evidence.

She looked around the room.

> "Good news."

Everyone waited.

> "There was no data breach."

Emma smiled with relief.

Sarah explained,

> "The customer saw an unfamiliar login notification."

> "It turned out to be their own login from a new mobile device."

Alex laughed.

> "Better to investigate a false alarm than ignore a real attack."

Sarah nodded.

> "Exactly."

Lessons Learned

Before ending the meeting, Maya summarized everything the team had practiced.

She wrote another checklist.

- ☒ Data Breach Investigation
- ☒ Digital Forensics
- ☒ Chain of Custody
- ☒ Forensic Imaging
- ☒ Cryptographic Hash
- ☒ SHA-256
- ☒ Audit Logs
- ☒ Non-Repudiation
- ☒ Timeline Analysis
- ☒ Data Classification
- ☒ Data Retention
- ☒ GDPR
- ☒ PCI DSS
- ☒ ISO 27001
- ☒ Compliance Audit

She smiled.

> "Today's investigation reminded us of something important."

Emma looked up.

> "What's that?"

Maya answered,

> "Security isn't only about stopping attacks."

She paused.

> "It's also about proving what did—or did not—happen."

Alex packed away his laptop.

> "Evidence matters."

Sarah nodded.

> "And protecting evidence is just as important as protecting systems."

The team left the conference room with a deeper appreciation for digital forensics, knowing that every log entry, every hash, and every timestamp could one day become a crucial piece of evidence in understanding a security incident.

Part 4: Cybersecurity

Chapter 6: Building Security into Every Release

A few weeks after the security incident, Maya called the engineering team into the conference room.

This time, there were no alerts.

No suspicious emails.

No malware.

No ongoing investigations.

Instead, Maya had a simple question.

> "When should security begin?"

Alex answered,

> "Before we deploy."

Sarah smiled.

> "Earlier."

Emma tried again.

> "During development?"

Sarah nodded.

> "Earlier."

The room became quiet.

Finally, James said,

> "The moment we start designing the system."

Sarah smiled.

> "Exactly."

DevSecOps

Maya wrote one word on the whiteboard.

DevSecOps

Emma asked,

> "How is that different from DevOps?"

Sarah explained,

> "DevOps integrates development and operations."

> "DevSecOps integrates security into every stage of the software lifecycle."

Alex nodded.

> "Security isn't a separate department anymore."

> "It's everyone's responsibility."

Shift Left Security

Sarah drew a timeline.

Design → Development → Testing → Deployment → Production

She pointed to the beginning.

> "This is called **Shift Left Security**."

David asked,

> "Why 'left'?"

Sarah smiled.

> "Because security moves earlier in the development process."

Emma nodded.

> "Finding a vulnerability during design is much cheaper than fixing it after deployment."

Threat Modeling

Before writing any code, the team gathered around the architecture diagram.

Sarah asked,

> "If you were an attacker, where would you start?"

Alex looked at the login service.

> "Authentication."

Emma pointed at the file upload feature.

> "I'd check whether uploads are validated."

Sarah smiled.

> "That's **threat modeling**."

She continued,

> "We identify possible attacks before attackers do."

Risk Assessment

Maya created a table.

Each identified threat received two scores.

* Likelihood

* Impact

James asked,

> "Why both?"

Sarah explained,

> "A highly likely issue with little impact isn't always our highest priority."

> "A rare issue with catastrophic impact might deserve immediate attention."

Emma nodded.

> "That's **risk assessment**."

Static Application Security Testing (SAST)

Alex committed new code.

Immediately, the CI pipeline started.

One stage failed.

Emma looked at the report.

> "The **SAST** scanner found a hardcoded password."

Sarah explained,

> "**Static Application Security Testing** analyzes source code without executing it."

Alex removed the hardcoded secret.

The pipeline passed.

Dynamic Application Security Testing (DAST)

Chris deployed the application to the staging environment.

Another scanner started running.

David asked,

> "Why scan it again?"

Sarah replied,

> "**Dynamic Application Security Testing** examines the running application."

Emma smiled.

> "So SAST analyzes code."

> "DAST analyzes behavior."

Software Composition Analysis

James opened the dependency list.

The application used hundreds of third-party libraries.

Sarah pointed to one package.

> "This version has a known vulnerability."

Alex asked,

> "But we didn't write that code."

Sarah nodded.

> "Exactly."

> "That's why we perform **Software Composition Analysis (SCA)**."

Secrets Management

Emma reviewed the configuration files.

She noticed an API key.

> "Should this be here?"

Sarah shook her head immediately.

> "Never commit secrets to Git."

Chris opened the company's **Secrets Manager**.

Passwords.

API keys.

Database credentials.

Encryption keys.

Everything was stored securely.

Secret Rotation

David asked,

> "If an API key is leaked, what do we do?"

Sarah replied,

> "We **rotate** the secret."

She explained,

> "Generate a new credential."

> "Revoke the old one."

> "Update every service that depends on it."

SBOM

The release process generated another document.

Emma looked curious.

> "What's an **SBOM**?"

Sarah answered,

> **"Software Bill of Materials"**.

She explained,

> "It's a complete inventory of every software component included in our application."

Alex smiled.

> "Like an ingredient list on food packaging."

> "Exactly."

Continuous Security

The CI/CD pipeline continued running.

Security checks appeared at every stage.

- * Dependency scanning
- * SAST
- * DAST
- * Container image scanning
- * Secret scanning

Emma smiled.

> "Security is happening automatically."

Sarah nodded.

> "That's **continuous security**."

Security Gates

The deployment stopped unexpectedly.

Alex frowned.

> "Another failure?"

Chris pointed at the report.

> "A critical vulnerability was detected."

Sarah smiled.

> "That's a **security gate**."

She continued,

> "The application cannot be deployed until the issue is resolved."

Emma nodded.

> "Better to delay a release than deploy an insecure application."

Security Champions

As the engineering organization grew, Sarah couldn't review every project herself.

Maya suggested a solution.

Each development team selected one engineer.

Emma volunteered.

Sarah smiled.

> "Congratulations."

> "You're now our team's **Security Champion**."

Emma laughed.

> "Does that mean I'm joining the security team?"

Sarah replied,

> "Not exactly."

> "It means you'll help your team make better security decisions."

A New Culture

Six months later, another penetration test was performed.

Daniel returned to evaluate the platform.

He smiled after reviewing the results.

> "This is one of the strongest engineering cultures I've seen."

Maya asked,

> "Because of our tools?"

Daniel shook his head.

> "The tools help."

He looked around the room.

> "But your biggest strength is your mindset."

Emma smiled.

> "Security isn't something we add."

Alex nodded.

> "It's something we build."

Looking Back

Before ending the meeting, Maya reviewed everything the team had learned over the past several months.

They had learned how to:

- * Recognize phishing attacks.
- * Protect accounts with MFA.
- * Investigate malware.
- * Secure application code.
- * Respond to incidents.
- * Perform digital forensics.
- * Build security into every release.

Sarah smiled.

> "Cybersecurity isn't about being afraid of attackers."

She looked around the room.

> "It's about building systems that remain trustworthy even when attackers try their hardest."

Final Lessons

Maya wrote one final checklist.

- ✅ DevSecOps
- ✅ Shift Left Security
- ✅ Threat Modeling
- ✅ Risk Assessment
- ✅ Static Application Security Testing (SAST)
- ✅ Dynamic Application Security Testing (DAST)
- ✅ Software Composition Analysis (SCA)
- ✅ Secrets Management

- ✔ Secret Rotation
- ✔ Software Bill of Materials (SBOM)
- ✔ Continuous Security
- ✔ Security Gates
- ✔ Security Champion

She smiled.

> "Technology changes every year."

Emma nodded.

> "Attackers change every day."

Sarah closed her laptop.

> "But one principle never changes."

Everyone looked toward the whiteboard.

****Build security into everything.****

The room became quiet.

Then everyone returned to work—not just as developers, operators, testers, and architects, but as engineers who understood that security is not a feature, a checklist, or a single tool.

It is a habit.

It is a mindset.

And it is a responsibility shared by everyone.